NAME : K.YOGANAND

REG.NO:192525173

COURSE CODE:CSA0604

COURSE NAME:DESIGN AND ANALYSIS OF ALGORITHMS

Session 2 - Lab Experiments

1) **Write a program that takes an input list of n numbers and creates a new list containingonly the unique elements from the original list. What is the space complexity of the algorithm?**

   CODE:

```
n = int(input("Enter the number of elements: "))

numbers = list(map(int, input("Enter the elements: ").split()))

unique = []

for num in numbers:
    if num not in unique:
        unique.append(num)

print("Original list:", numbers)
print("Unique elements:", unique)
```

OUTPUT:

```
Enter the number of elements: 8
Enter the elements: 10 20 10 30 20 40 30 50
Original list: [10, 20, 10, 30, 20, 40, 30, 50]
Unique elements: [10, 20, 30, 40, 50]
```

2) Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

CODE:

```
#include <stdio.h>

int main() {
   int n, i, j, temp;
   int a[100];

   printf("Enter the number of elements: ");
   scanf("%d", &n);

   printf("Enter %d elements: ", n);
   for (i = 0; i < n; i++) {
      scanf("%d", &a[i]);
   }

   // Bubble Sort
   for (i = 0; i < n - 1; i++) {
      for (j = 0; j < n - i - 1; j++) {
         if (a[j] > a[j + 1]) {
            temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
         }
      }
   }
```

```
    printf("Sorted array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }

    return 0;
}
```

OUTPUT:

```
Enter the number of elements: Enter 6 elements: Sorted array: 1 3 5 8 12 19
```

3) Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

CODE:

```
#include <stdio.h>

int main() {
   int arr[100], n, x;
   int low, high, mid;
   int found = 0;

   printf("Enter the number of elements: ");
   if (scanf("%d", &n) != 1) return 1;

   printf("Enter %d elements in sorted order: ", n);
   for (int i = 0; i < n; i++) {
      if (scanf("%d", &arr[i]) != 1) return 1;
   }

   printf("Enter the number to search: ");
   if (scanf("%d", &x) != 1) return 1;

   low = 0;
   high = n - 1;

   while (low <= high) {
      mid = (low + high) / 2;

      if (arr[mid] == x) {
         found = 1;
         break;
      }
      else if (arr[mid] < x) {
         low = mid + 1;
      }
      else {
         high = mid - 1;
      }
   }

   if (found)
      printf("%d exists in the array.\n", x);
   else
```

```
        printf("%d does not exist in the array.\n", x);

    return 0;
}
```

OUTPUT:

```
Enter the number of elements: Enter 5 elements in sorted order: Enter the number to search: 40 exists in the array.
```

4) Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in O(nlog(n)) time complexity and with the smallest space complexity possible.

CODE:

```
#include <stdio.h>

// Merge two sorted parts
void merge(int nums[], int left, int mid, int right) {
    int i = left;
    int j = mid + 1;
    int k = 0;

    int temp[100];

    while (i <= mid && j <= right) {
        if (nums[i] <= nums[j]) {
            temp[k] = nums[i];
            i++;
        } else {
            temp[k] = nums[j];
            j++;
        }
        k++;
    }

    while (i <= mid) {
        temp[k] = nums[i];
        i++;
        k++;
    }

    while (j <= right) {
        temp[k] = nums[j];
        j++;
        k++;
    }
```

```
    for (i = left, k = 0; i <= right; i++, k++) {
        nums[i] = temp[k];
    }
}

// Merge Sort
void mergeSort(int nums[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;

        mergeSort(nums, left, mid);
        mergeSort(nums, mid + 1, right);

        merge(nums, left, mid, right);
    }
}

int main() {
    int nums[100];
    int n, i;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    mergeSort(nums, 0, n - 1);

    printf("Sorted array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", nums[i]);
    }

    return 0;
}
```

OUTPUT:

```
Enter the number of elements: Enter 6 elements: Sorted array: 1 2 7 9 11 15
```